

Eng. da Computação



Hashing na Cibersegurança: Principais características

Prof. Dr. Ruy de Oliveira

O QUE É HASHING?

- Técnica de criptografia baseada em uma função matemática
- Processo unidirecional: converte dados em uma sequência de caracteres (hash)
- Não permite a reversão para obter os dados originais (não é decodificável)

Finalidade:

- Protege informações sensíveis como
 - Documentos
 - Senhas
 - Mensagens
- Utilizado também em:
 - Recuperação de dados
 - Criptografia

Componentes do Hashing

Componente	Função
Chave de entrada	Dados ou mensagem processados para gerar o hash
Função hash	Algoritmo que realiza a conversão dos dados
Tabela hash	Estrutura que armazena os dados e mapeia chaves para valores

HASHING

- Vivemos em uma sociedade altamente digitalizada. Todos os dias realizamos atividades como:
 - acessar redes sociais
 - efetuar pagamentos via PIX
 - utilizar aplicativos bancários
 - enviar mensagens pelo WhatsApp
 - realizar compras pela Internet
 - baixar softwares
 - armazenar documentos na nuvem



A resposta está em uma técnica chamada **hashing**.

Como podemos garantir que um arquivo não foi alterado?

Por exemplo, imagine que você baixe uma imagem ISO do Ubuntu Linux.

Como saber se:

- o download ocorreu corretamente?
- nenhum bit foi corrompido?
- um invasor não modificou o arquivo?

O QUE É O HASHING?

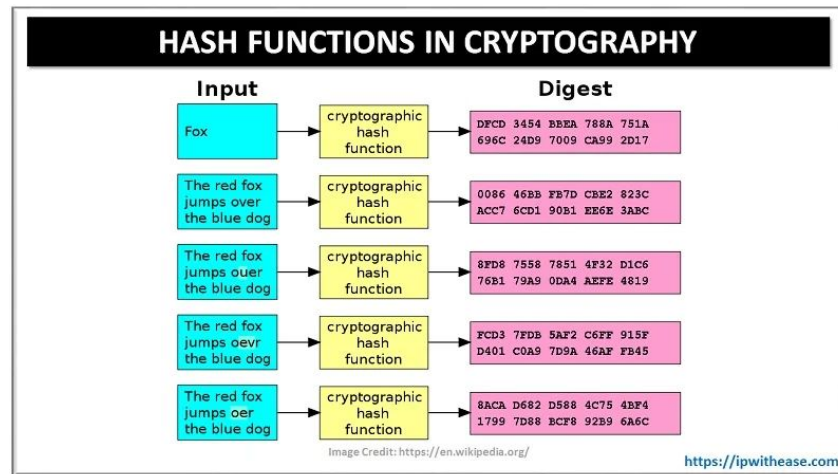
- É um dos conceitos fundamentais da Segurança da Informação e Criptografia
- Uma função hash é um algoritmo matemático que recebe uma entrada de qualquer tamanho e gera uma saída de tamanho fixo, chamada de:
 - Hash
 - Resumo criptográfico (digest)
 - Impressão digital dos dados

Fluxo do processo:

Entrada → Função Hash → Hash (tamanho fixo)

Exemplo com SHA-256:

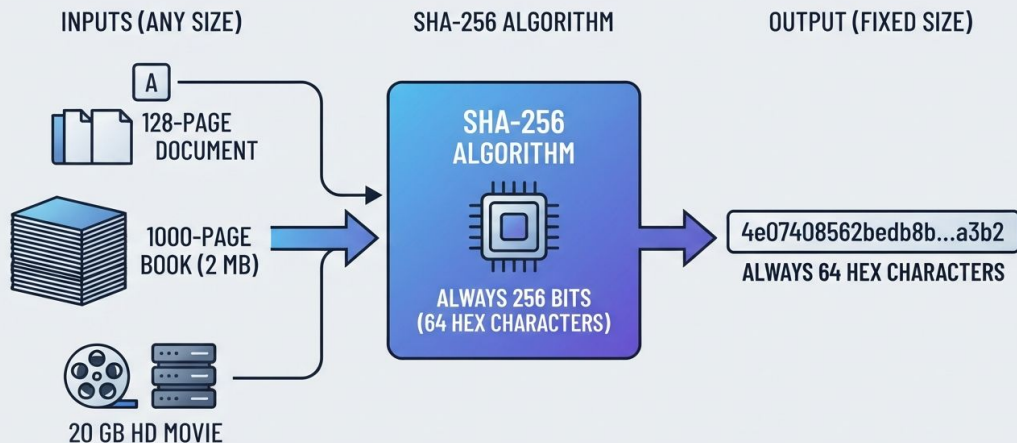
Entrada: Olá Mundo → Saída: 33cf6123dd5c46d7b6fdc9cd72abbf66...



CARACTERÍSTICA DO TAMANHO FIXO

- O popular algoritmo **SHA-256** sempre retorna 256 bits (64 caracteres hexadecimais), independentemente da entrada:

SHA-256: HASHING VARIABLE INPUTS TO A FIXED OUTPUT



- Uma letra
- Uma palavra
- Um livro inteiro
- Um filme de 20 GB
- Um sistema operacional completo

PARA QUE SERVE O HASHING?

QUATRO PRINCIPAIS USOS DE HASH EM SEGURANÇA



VERIFICAR INTEGRIDADE E IDENTIFICAR ARQUIVOS

Baixando ISO...



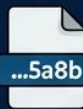
Hash oficial: ↓ Hash do arquivo:

...5a8b

VS

...5a8b

ARQUIVO ÍNTEGRO ✓



Diferentes

ARMAZENAR SENHAS COM TOTAL SEGURANÇA



BANCO DE DADOS HASH

Armazena apenas Hashes
Acesso negado sem hash original



Protegido

ASSINAR DOCUMENTOS E AUTENTICAR MENSAGENS



CONTRATO DIGITAL



MENSAGENS AUTÊNTICAS



ASSINADO

Gera Hash Único



O FUNCIONAMENTO DAS BLOCKCHAINS



IMUTÁVEL

Hash do Bloco Anterior

Dados de transação



Corrente de Hashes

ANALOGIA SIMPLES: EFEITO AVALANCHE

Documento → Função Hash → **Impressão Digital**

- Assim como duas pessoas dificilmente têm a mesma impressão digital, dois arquivos diferentes quase nunca geram o mesmo hash
- Qualquer pequena alteração no conteúdo cria uma impressão completamente diferente
 - **Efeito Avalanche !!!**

Exemplo:

Entrada

Computação

Computação

Hash SHA-256 (resumido)

fa574...

bc1d...

Conclusão: Mesmo alterando apenas um caractere (sem acento), o hash muda totalmente — esse comportamento é chamado de Efeito Avalanche

PROPRIEDADES DE UMA BOA FUNÇÃO HASH



REPRESENTAÇÃO HEXADECIMAL

- Os algoritmos geram sequências de bits (10101100...), difíceis de ler
- Usamos a representação hexadecimal (base 16) para simplificar:
 - Valores de 0 a 9 e de A a F
 - Cada caractere representa 4 bits
- Exemplo: $1111 = F \rightarrow$ reduz o tamanho visual do hash pela metade

PRIMEIRO PROGRAMA EM PYTHON

Usamos a biblioteca nativa `hashlib`:

python

Run

```
import hashlib

texto = "Olá Mundo"
hash_obj = hashlib.sha256(texto.encode())
print(hash_obj.hexdigest())
```

Saída: 33cf6123dd5c46d7b6fdc9cd72abbbf66...

TESTE EM PYTHON

```
import hashlib

texto = "Olá Mundo"
hash_obj = hashlib.sha256(texto.encode())
print(hash_obj.hexdigest())
```

Para resultado em binário:

```
binary_string = bin(int(hash_obj.hexdigest(), 16))[2:].zfill(256)
print(binary_string)
```

Saída: 360a066e1398f009cf5a1c2eb2a508d5a7a076d076edcbc685e780b22d385569b

`import hashlib`: Carrega a biblioteca de funções hash

`texto.encode()`: Converte o texto para bytes (funções hash só trabalham com bytes)

`hashlib.sha256(...)`: Aplica o algoritmo SHA-256

`.hexdigest()`: Converte o resultado para formato hexadecimal legível

COMPARANDO VÁRIAS ENTRADAS

```
import hashlib

palavras = ["Python", "python", "PYTHON",
            "Python!"]
for p in palavras:
    h = hashlib.sha256(p.encode()).hexdigest()
    print(f"{p} → {h[:8]}...")
```

Saída:

Python → 18885f27...
python → 11a4a60b...
PYTHON → 329b3dcf...
Python! → 70d8fd4f...

→ Maiúsculas, minúsculas e símbolos alteram completamente o hash

DEMONSTRAÇÃO DO EFEITO AVALANCHE

```
import hashlib  
t1, t2 = "computador", "Computador"  
h1 = hashlib.sha256(t1.encode()).hexdigest()  
h2 = hashlib.sha256(t2.encode()).hexdigest()  
  
print(f"{t1} → {h1[:8]}...")  
print(f"{t2} → {h2[:8]}...")
```

Saída:

computador → 0c0ae2d9...

Computador → 35cd6bf4...

→ Apenas a primeira letra mudou, mas os hashes são totalmente diferentes

HASH DE ARQUIVOS

```
import hashlib

sha = hashlib.sha256()
with open("/home/ruy/Documents/Microcontroller.pdf",
"rb") as arq:
    while bloco := arq.read(4096):
        sha.update(bloco)

print(sha.hexdigest())
```

Saída:

6cf454a773db3adf310d63f1f1ec6275e3ed0c6777c0e77c3940f6238cd6b307

"rb": Abre o arquivo em modo leitura binário

Lê o arquivo em blocos de 4KB para não sobrecarregar a memória

Por que Ler em Blocos?

Para arquivos muito grandes (ex: vídeo de 100 GB), carregar tudo na memória é inviável.

Funcionamento:

1. Lê 4096 bytes do arquivo
2. Atualiza o hash com esse trecho
3. Repete até o final do arquivo

Permite processar arquivos gigantes com pouca memória RAM

HASH DE ARQUIVOS

```
import hashlib

# Hash esperado (do site oficial)
HASH_ESPERADO = "0816c4761c97ecdb3f50a3924de0a93fd78cb63ee8e6c04201ddfaedca500b0b"
NOME_ARQUIVO = "/home/ruy/Downloads/Python-3.12.13.tgz"

# Calcula o hash do arquivo
sha = hashlib.sha256()
with open(NOME_ARQUIVO, "rb") as arq:
    while bloco := arq.read(4096):
        sha.update(bloco)

hash_calculado = sha.hexdigest()

# Exibe os resultados
print(f"Hash esperado: {HASH_ESPERADO}")
print(f"Hash calculado: {hash_calculado}")
print("-" * 70)

# Verifica
if hash_calculado == HASH_ESPERADO:
    print("✅ ARQUIVO ÍNTEGRO! Pode usar com segurança.")
else:
    print("❌ ARQUIVO CORROMPIDO OU MODIFICADO! Não use!")
```

1. O site oficial disponibiliza o hash esperado do arquivo
2. Você calcula o hash do arquivo baixado
3. Compara os dois valores:

Usado em: Downloads de ISOs (Ubuntu, Windows), backups, atualizações.

ARMAZENAMENTO SEGURO DE SENHAS

✗ Nunca armazene senhas em texto puro! → Se o banco de dados for roubado, todas as contas estarão comprometidas

✓ Armazene apenas o hash da senha → Ninguém tem acesso à senha original, mesmo que o banco seja invadido

Usuário	Hash (seguro)
---------	---------------

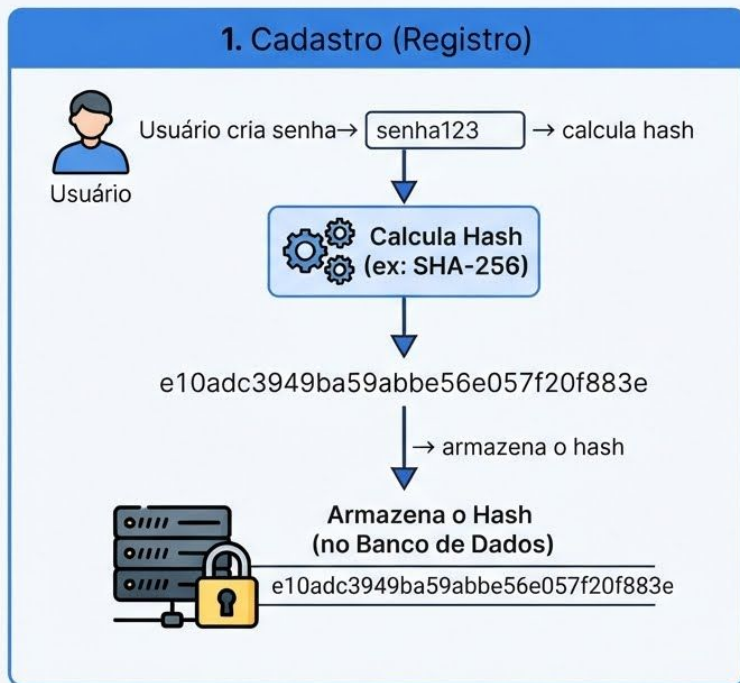
João	a73f91...
------	-----------

Maria	c911b2...
-------	-----------

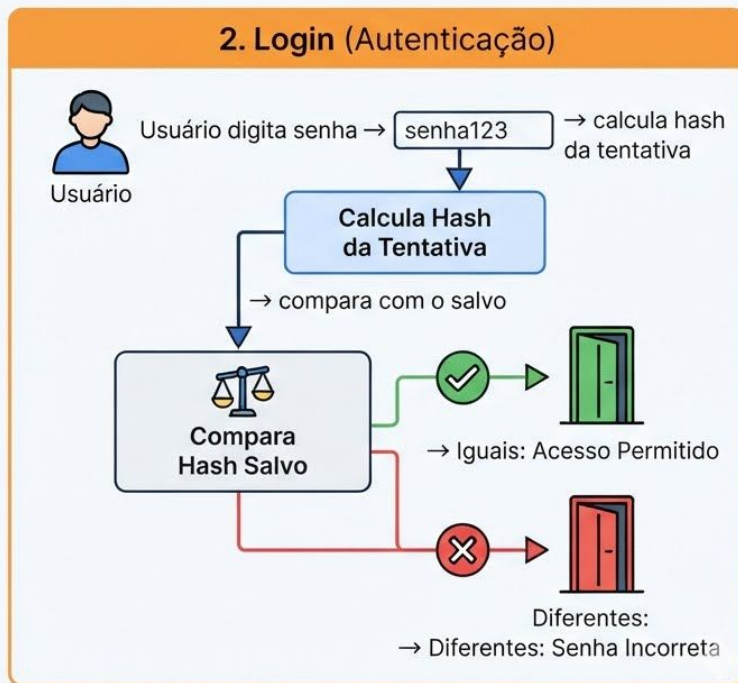
LOGIN SEGURO

Como Funciona o Login?

1. Cadastro (Registro)



2. Login (Autenticação)



EXEMPLO BÁSICO DE CADASTRO E LOGIN

```
import hashlib

# Cadastro
senha = input("Crie sua senha: ")
hash_salvo = hashlib.sha256(senha.encode()).hexdigest()

# Login
tentativa = input("Digite a senha: ")
hash_tentativa = hashlib.sha256(tentativa.encode()).hexdigest()

if hash_tentativa == hash_salvo:
    print("✅ Acesso permitido!")
else:
    print("❌ Senha incorreta!")
```

Atualize o código para armazenar as "senhas" em um arquivo, e posteriormente ler do arquivo.

PROBLEMAS — ATAQUE POR DICIONÁRIO

- Apenas o SHA-256 não é suficiente para senhas!
- Invasores usam listas de milhões de senhas comuns (123456, senha, admin), calculam seus hashes e comparam com os dados roubados
- Como o SHA-256 é muito rápido, bilhões de tentativas podem ser feitas por segundo com GPUs



Rainbow Tables

- São tabelas prontas com milhões/bilhões de pares Senha → Hash
- Permitem descobrir senhas fracas em segundos, sem precisar calcular cada hash

SOLUÇÃO — USAR A ESTRATÉGIA DO SALT

- Salt é um valor aleatório único gerado para cada usuário
- Em vez de $\text{SHA-256}(\text{senha})$, usamos: $\text{SHA-256}(\text{senha} + \text{salt})$
- Vantagens:
 - Mesma senha $\rightarrow$ hash diferente para cada usuário
 - Torna Rainbow Tables inúteis
- O Sistema agora armazena: senha + salt

EXEMPLO COM SALT EM PYTHON

```
import hashlib
import secrets

senha = "minhasenha123"
salt = secrets.token_hex(16) # Gera salt aleatório de 16 bytes

dados = senha + salt
hash_final = hashlib.sha256(dados.encode()).hexdigest()

print(f"Salt: {salt}")
print(f"Hash: {hash_final}")
```

→ Cada execução gera um salt e um hash diferentes!

IMPACTO DAS ITERAÇÕES COM O ALGORITMO BCRIPT

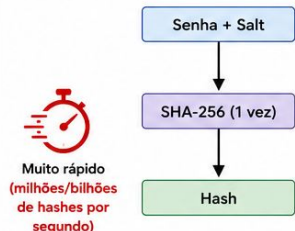
POR QUE **ITERAÇÕES (CUSTO)** TORNAM SUAS SENHAS **MAIS SEGURAS**

Algoritmos como bcrypt aplicam um custo computacional alto e ajustável.
Isso torna o cálculo do hash intencionalmente lento para o atacante e muito mais caro.



SEM ITERAÇÕES (SHA-256)

Muito rápido e barato para o atacante



Muito rápido
(milhões/bilhões
de hashes por
segundo)

IMPACTO PARA O ATACANTE

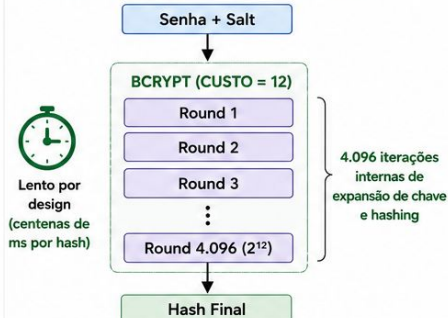
Cada senha testada exige apenas 1 hash.



Permite testar milhões ou bilhões de senhas por segundo com hardware especializado (GPUs, ASICs).

COM BCRIPT (EX.: CUSTO = 12)

Intencionalmente lento e caro para o atacante



Lento por design
(centenas de ms por hash)

4.096 iterações
internas de
expansão de chave
e hashing

IMPACTO PARA O ATACANTE

Cada senha testada exige milhares de operações de hash.



Reduz drasticamente a quantidade de senhas que podem ser testadas por segundo. O custo computacional aumenta exponencialmente conforme o valor do custo (ex.: 10, 12, 14...).

VANTAGENS DO BCRIPT (COM ITERAÇÕES)



1. PROTEÇÃO CONTRA FORÇA BRUTA E DICIONÁRIO

Cada tentativa de senha é muito mais cara para o atacante, dificultando ataques offline.



2. CUSTO AJUSTÁVEL (FATOR DE CUSTO)

Você pode aumentar o custo (ex.: 10, 12, 14) conforme o hardware evolui, mantendo suas senhas sempre protegidas.



3. BAIXO IMPACTO PARA O USUÁRIO

O atraso no login é pequeno (geralmente 100-300 ms), imperceptível para o usuário legítimo.



4. SEGURO MESMO COM BANCO DE DADOS VAZADO

Mesmo que o atacante obtenha o hash e o salt, precisará de muito tempo e recursos para testar cada senha.



5. PADRÃO RECOMENDADO

bcrypt é amplamente adotado e considerado seguro para armazenamento de senhas.

EXEMPLO NUMÉRICO DE IMPACTO PARA O ATACANTE

SEM ITERAÇÕES (SHA-256)

Suponha que o hardware consiga calcular 1.000.000 de hashes por segundo.



1.000.000
senhas testadas
por segundo

VS

COM BCRIPT (CUSTO = 12)

Com custo 12, cada senha leva ~100-300 ms para ser calculada (dependendo do hardware).



3 - 10
senhas testadas
por segundo

★ A diferença pode ser de 100.000x ou mais na quantidade de senhas testadas por segundo!

COMO FUNCIONA O BCRIPT



RESULTADO: bcrypt usa iterações (custo) para tornar o ataque caro e demorado, mas mantém a experiência do usuário rápida e segura.



Dica: Use o maior custo que sua aplicação tolerar (ex.: 12 ou mais) e aumente ao longo do tempo.

QUAL ALGORITMO USAR PARA SENHAS?

SHA-256 é ótimo para integridade, mas não ideal para senhas por ser muito rápido. Use algoritmos lentos (propositadamente) e seguros:

Algoritmo	Recomendado para senhas?
MD5	✗ Obsoleto, inseguro
SHA-1	✗ Obsoleto, inseguro
SHA-256	⚠ Apenas para integridade de arquivos/dados
PBKDF2	✓ Aceitável (padrão NIST; use $\geq 100k$ iterações + SHA-256)
bcrypt	✓ Sim, amplamente usado e seguro
scrypt	✓ Sim, exige mais memória e dificulta ataques com GPUs
Argon2	✓ Melhor opção atual (vencedor da competição Password Hashing Competition)

O PBKDF2 é uma opção confiável, mas se tiver liberdade para escolher, prefira Argon2 ou bcrypt: eles são mais resistentes a ataques modernos com placas de vídeo e circuitos especializados.

EXEMPLO: SALT COM PBKDF2

```
import hashlib
import secrets

senha = b"MinhaSenhaSegura123"

# Gera Salt único e seguro
salt = secrets.token_bytes(16) # 16 bytes = 128 bits

# Aplica PBKDF2 com SHA-256 e 150.000 iterações
hash_final = hashlib.pbkdf2_hmac(
    hash_name = "sha256",          # Algoritmo base
    password = senha,              # Senha do usuário
    salt = salt,                   # Salt único
    iterations = 150000            # Número de repetições
)

# Converte para hexadecimal para armazenar
hash_hex = hash_final.hex()
salt_hex = salt.hex()

print(f"Salt: {salt_hex}")
print(f"Hash PBKDF2: {hash_hex}")
```

Configuração mínima recomendada:

- Algoritmo base: SHA-256
- Mínimo de 100.000 iterações
- Salt seguro de 16 bytes ou mais

Como verificar no login?

```
# Dados recebidos do banco de dados
salt_armazenado = bytes.fromhex(salt_hex)
hash_armazenado = bytes.fromhex(hash_hex)

# Senha digitada pelo usuário no login
tentativa = b"MinhaSenhaSegura123"

# Recalcula o hash com os mesmos parâmetros
hash_tentativa = hashlib.pbkdf2_hmac(
    "sha256",
    tentativa,
    salt_armazenado,
    150000
)

# Compara os resultados
if hash_tentativa == hash_armazenado:
    print("✅ Acesso permitido!")
else:
    print("❌ Senha incorreta!")
```

EXEMPLO: SALT COM BCrypt

```
import bcrypt

# Senha do usuário (deve estar no formato de bytes)
senha_usuario = b"MinhaSenhaSegura123@"

# 1. Gera um Salt ALEATÓRIO e ÚNICO + cria o hash
seguro
# O valor 12 = fator de custo (quanto maior, mais lento
e seguro)
hash_armazenado = bcrypt.hashpw(
    senha_usuario,
    bcrypt.gensalt(rounds=12)
)

# Converte para texto para salvar no banco de dados
hash_texto = hash_armazenado.decode("utf-8")

print("Hash completo para armazenar:")
print(hash_texto)
print("\n⚠ O Salt já está embutido nesse valor! Não
precisa salvá-lo separadamente.")
```

Como verificar no login?

```
# Recupera o hash salvo do banco de dados
(converta de volta para bytes)
hash_salvo = hash_texto.encode("utf-8")

# Senha que o usuário digitou no login
senha_tentativa = b"MinhaSenhaSegura123@"

# Compara a tentativa com o hash salvo
(bcrypt extrai o Salt sozinho)
if bcrypt.checkpw(senha_tentativa,
    hash_salvo):
    print("✅ Acesso permitido! Senha
confirmada.")
else:
    print("❌ Senha incorreta ou usuário não
encontrado.")
```

📌 Vantagens desse método

- ✅ Salt automático: não precisa gerenciar ou armazenar o Salt separadamente
- ✅ Fator de custo ajustável: você pode aumentar a segurança conforme os computadores ficam mais rápidos
- ✅ Resistente a GPUs: dificulta ataques de força bruta em larga escala
- ✅ Simples e seguro: segue as recomendações da OWASP e NIST

O FATOR DE CUSTO DO BCRYPT

- Controlado por rounds em gensalt(rounds=X): cada número dobra o tempo de cálculo
- Recomendações:
 - Mínimo: 10 (~100ms por senha)
 - Ideal para 2026: 12 (~400ms por senha)
 - Aumente o valor conforme computadores ficarem mais rápidos

APLICAÇÕES DO HASHING NO DIA A DIA

- Armazenamento seguro de senhas
- Verificação de integridade de arquivos
- Assinaturas e certificados digitais
- Blockchain e criptomoedas
- Controle de versões (Git)
- Perícia computacional

CONCLUSÕES

- Hashing cria uma **impressão digital única** para qualquer dado
- Use SHA-256 para integridade; para senhas: **PBKDF2, bcrypt, scrypt ou Argon2 + Salt único**
- O Salt pode ser armazenado publicamente — sua segurança vem da aleatoriedade, não do sigilo
- É a base para quase todos os sistemas seguros modernos

FERRAMENTAS ÚTEIS

- <https://emn178.github.io/online-tools/sha256.html>
- <https://www.md5hashgenerator.com/>
- <https://bcrypt.online/>
- <https://caddy.tech/tools/hash-generator>

LINKS INTERESSANTES

- <http://www.cse.yorku.ca/~oz/hash.html>
- <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-4-hashing/>
- <https://cp-algorithms.com/string/string-hashing.html>
- <https://www.codecademy.com/resources/blog/what-is-hashing>
- <https://www.crowdstrike.com/en-us/cybersecurity-101/data-protection/data-hashing/>
- https://medium.com/@0x_AJ/hashing-in-cybersecurity-a-beginners-guide-to-digital-fingerprints-85ab6033dffb

BIBLIOGRAFIA BÁSICA

- MARTINS, N. A. Sistemas Microcontrolados. 1a ed. São Paulo: Novatec, 2005.
- PEREIRA, F. Microcontroladores PIC - Programação em C. 2a ed. São Paulo: Érica, 2003.